Technical Design Document

Motadata

> 4 AWS Programs

> 9 AWS Competencies

> 19 AWS Service Delivery Programs

Revision History

Version	Date	Description	Author	Reviewer
v0.1	15-07-26	Initial Draft	Vynkatesh Sinhasane	Aditya Kapole Ankith S
v0.2	27-07-26	Updated with the details of the Investigator tool.	Vynkatesh Sinhasane	Aditya Kapole Ankith S

Table of Contents

1. Introduction

This document describes the technical design for the Motadata AI-Powered Root Cause Analysis Agent, a GenAI solution built on Amazon Bedrock AgentCore and LangGraph. The system delivers automated RCA capabilities by correlating telemetry data (metrics, logs, and traces) across dependent services to identify root causes and surface supporting evidence within Motadata's observability platform.

1.1. Key Objectives

1.1.1. Implement an automated multi-agent RCA system using Amazon Bedrock AgentCore as the serverless execution backbone with LangGraph for agent orchestration.

1.1.2. Correlate telemetry data across metrics, logs, and traces to identify root causes with an accuracy of **85% or higher**.

1.1.3. Deliver responses within **60 seconds or less** per query.

1.1.4. Demonstrate **multi-tenant isolation**.

1.1.5. Implement comprehensive security, monitoring, and observability controls, including IAM least-privilege access, KMS encryption, Bedrock Guardrails, AWS WAF protection, logging, auditing, and continuous monitoring.

2. Background

This section provides context on the business drivers, success criteria, and constraints that shaped the technical design of the RCA Agent.

2.1. Business Requirements

Motadata (Mindarray Systems Pvt Ltd) operates in the IT Infrastructure Monitoring and Observability sector, providing a unified platform that combines IT Service Management (ITSM) capabilities with deep telemetry monitoring across metrics, logs, flow data, and traces. This GenAI ~~Proof of Concept~~ solution enhances the existing observability platform with agentic AI-powered root cause analysis using Amazon Bedrock AgentCore and LangGraph orchestration. The engagement validates the feasibility, performance, and cost-effectiveness of delivering automated RCA capabilities to reduce mean time to resolution.

2.1.1. Primary Use Cases:

2.1.1.1. **Automated Root Cause Analysis:** Correlate telemetry data across metrics, logs, and traces to identify root causes of incidents and surface supporting evidence with remediation recommendations.

2.1.1.2. **Infrastructure Monitoring & Security:** Query live CloudWatch alarms, Security Hub findings, and Trusted Advisor recommendations to assess and validate tenant infrastructure health.

2.1.1.3. **Operational Intelligence:** Provide cost analysis, incident ticket management, and documentation retrieval to support day-to-day MSP operations.

2.1.2. Technical Requirements:

2.1.2.1. Multi-agent orchestration using LangGraph with a supervisor pattern routing to specialist agents via Agent-to-Agent (A2A) protocol.

2.1.2.2. Serverless execution on Amazon Bedrock AgentCore Runtime with microVM isolation per session.

2.1.2.3. Session persistence using AgentCore Memory with SemanticFacts and SessionSummaries for cross-session context.

2.1.2.4. PostgreSQL database (RDS) for structured telemetry storage with tenant-isolated schemas.

2.1.2.5. React frontend served via CloudFront with Cognito authentication and DynamoDB-backed chat history.

2.2. Project Success Criteria

2.2.1. All required AWS services (Bedrock AgentCore, LangGraph, Claude Sonnet 4.5, ECS, API Gateway, DynamoDB, S3) are configured and operational.

2.2.2. RCA agent achieves root cause identification accuracy of 90% or higher against client-provided telemetry datasets (18/20 test queries pass).

2.2.3. Response time remains within 30-55 seconds per invocation under warm conditions and under 70 seconds on cold start.

2.2.4. Multi-tenant isolation is validated with Cognito JWT, namespaced memory, and STS role assumption per customer account.

2.2.5. End-to-end RCA workflow including telemetry ingestion, multi-step reasoning, tool invocations, and output generation is tested and approved.

2.2.6. Knowledge transfer sessions are conducted, and comprehensive documentation is delivered.

2.3. Assumptions & Dependencies

2.3.1. Assumptions:

2.3.1.1. Telemetry data is extracted from tenant infrastructure in structured format (PostgreSQL) covering anomaly patterns and RCA scenarios across metrics, logs, and traces.

2.3.1.2. AWS accounts should have all required service quotas and Bedrock model access enabled.

2.3.1.3. The solution environment operates as a single-region deployment; multi-region is out of scope.

2.3.1.4. Demo data represents realistic observability scenarios sufficient to validate RCA accuracy at 90%.

2.3.1.5. Client team is available for functional testing and acceptance within the 6-week project timeline.

2.3.1.6. Internet connectivity is available for CloudFront distribution and API Gateway access.

2.3.2. Dependencies:

2.3.2.1. Amazon Bedrock AgentCore service availability in the ap-south-1 region with support for LangGraph-based runtimes.

2.3.2.2. Anthropic Claude Sonnet 4.5 and Claude Haiku 4.5 model access enabled in the Bedrock console.

2.3.2.3. AgentCore Memory, Gateway, and Observability features in General Availability status.

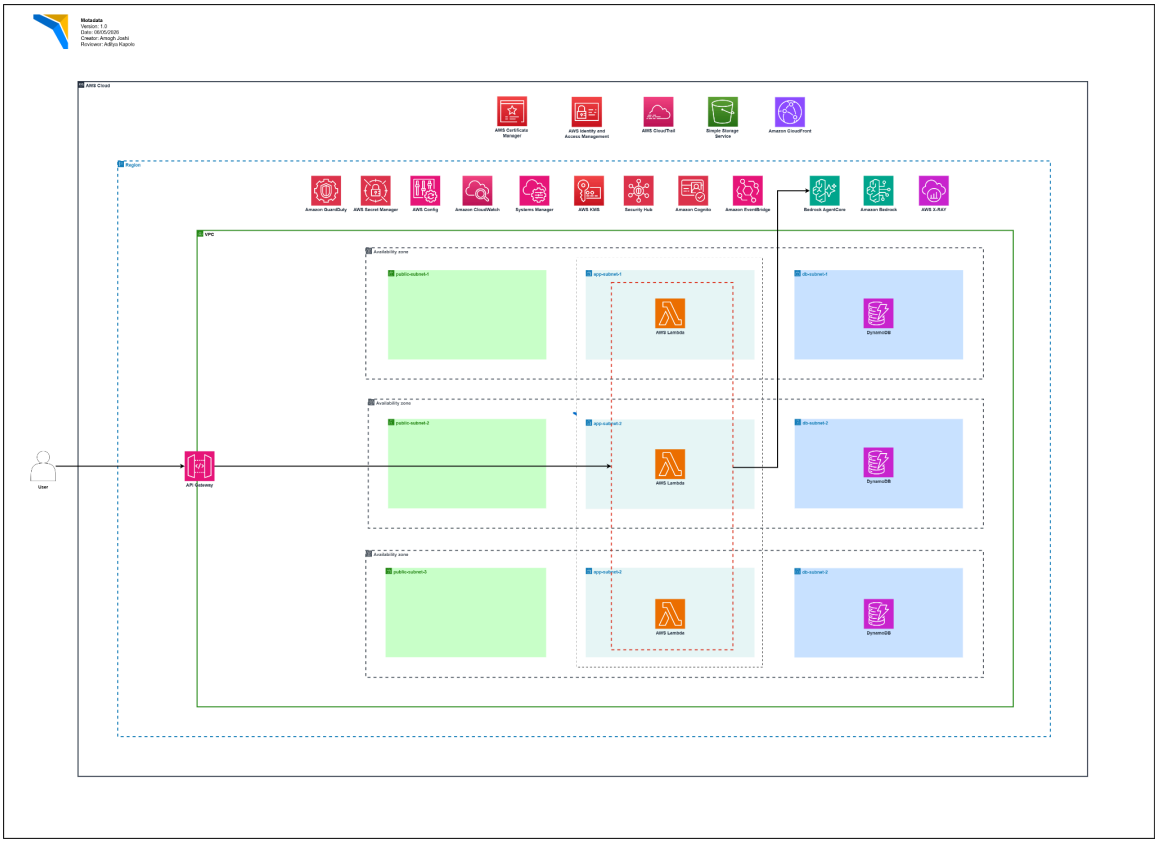
2.3.2.4. Client-provided telemetry datasets covering at least 20 RCA scenarios for accuracy validation.

2.3.2.5. AWS CDK and AgentCore CLI tooling compatibility with the deployed runtime configurations.

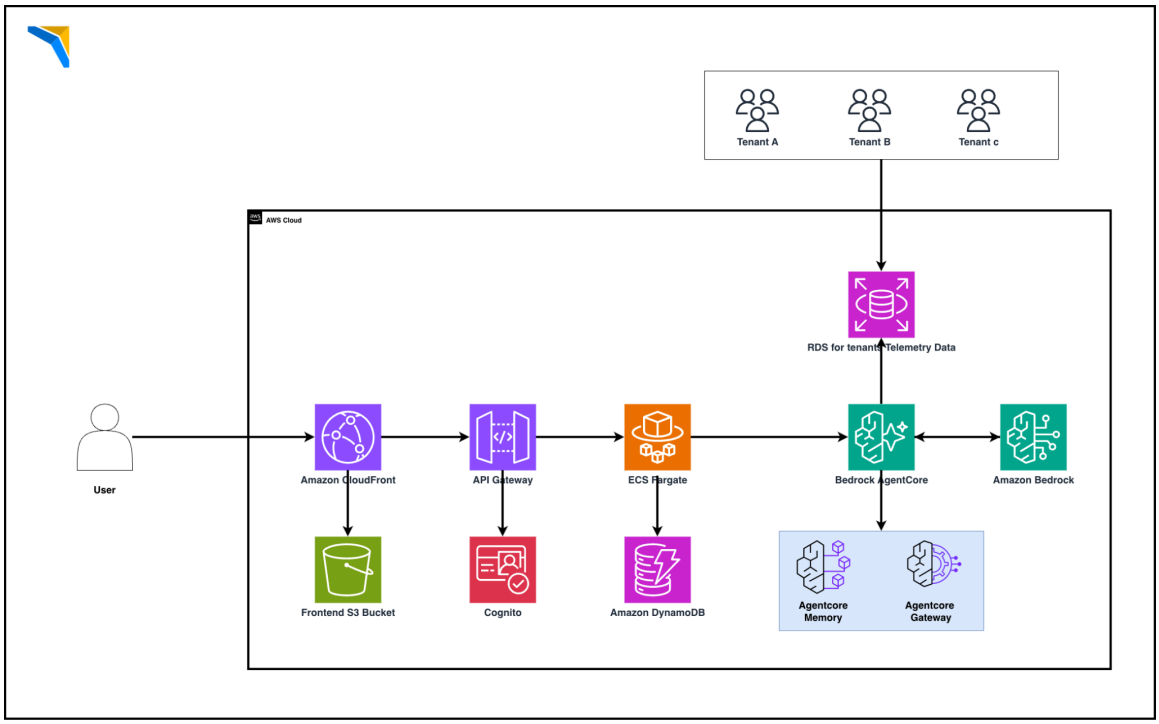
2.3.2.6. Cognito User Pool pre-configured with test users for authentication flow validation.

3. Solution Architecture

3.1. High-Level Diagram



3.2. Flow Diagram



3.3. Architecture Principles

3.3.1. Multi-Agent Orchestration

3.3.1.1. Supervisor-Based Routing: A central supervisor agent classifies user intent, routes requests to the appropriate specialist agent using the A2A protocol, and aggregates responses for multi-domain queries.

3.3.1.2. Specialist Agents: Each specialist agent operates independently with its own LangGraph workflow, dedicated tools, and system prompts, enabling domain-specific reasoning and execution.

3.3.1.3. Serverless Execution: All agents run on Bedrock AgentCore Runtime with automatic scaling and dedicated microVM isolation to ensure secure, infrastructure-free, multi-tenant execution.

3.3.1.4. Tool Access: AgentCore Gateway exposes CloudWatch, AWS APIs, and AWS Knowledge through MCP, providing standardized and secure tool invocation using SigV4 and OAuth authentication.

3.3.2. Separation of Concerns

3.3.2.1. **Layered Architecture:** The presentation layer (React + CloudFront) manages user interactions, the application layer (ECS FastAPI) handles authentication and request processing, and the AgentCore layer performs AI reasoning, memory management, and tool execution.

3.3.2.2. **Data Isolation:** Tenant data is isolated through tenant-specific database partitioning, dedicated memory namespaces, and STS AssumeRole-based access to ensure secure, tenant-scoped AWS resource access.

3.3.3. Resilience & Observability

3.3.3.1. **Fault Tolerance:** AgentCore automatically retries failed agent invocations, ECS maintains healthy application instances through health checks, and DynamoDB provides built-in high availability across multiple Availability Zones.

3.3.3.2. **Monitoring & Tracing:** AWS X-Ray with ADOT enables end-to-end request tracing, AgentCore Observability captures agent and tool execution details, and CloudWatch Metrics and Alarms provide real-time monitoring and alerting.

4. Data Architecture

The data architecture supports multi-tenant telemetry storage, session persistence, and agent memory. Storage is distributed across S3 for raw data and assets, DynamoDB for session state, and RDS PostgreSQL for structured telemetry queries.

4.1. S3 Bucket Architecture

Bucket	Purpose	Key Structure
Frontend Bucket	React 18 SPA hosting (frontend)	index.html, assets/js/, assets/css/
Telemetry Data Bucket	Telemetry data and agent assets	{tenant_id}/telemetry/
Agent Artifacts Bucket	Agent runtime container packages	runtime/{agent_name}/

All buckets are configured with versioning enabled, SSE-KMS encryption using AWS managed keys, and bucket policies restricting access to authorized IAM roles only. Public access is blocked on all buckets except the frontend assets bucket, which is accessed exclusively through the CloudFront Origin Access Identity.

4.2. DynamoDB Table Design

DynamoDB serves as the persistence layer for chat session history and request tracking. The table design optimizes for the primary access pattern of retrieving conversation history by user and session.

Table	Partition Key	Sort Key	Purpose	Capacity Mode

Chat-requests	request_id (S)	timestamp (S)	Chat request/response state, streaming events	On-Demand

4.3. RDS PostgreSQL Schema

The table uses on-demand capacity mode to accommodate variable solution workloads without capacity planning. Encryption at rest is enabled using AWS-managed keys. TTL is configured on the expires_at attribute to automatically purge sessions older than 30 days.

Column	Type	Description
tenant_id	TEXT NOT NULL	Tenant identifier for multi-tenant isolation
ts	TIMESTAMPTS NOT NULL	Log event timestamp
stream	TEXT NOT NULL	Log source: payment_service_logs, kernel_logs, infra_change_logs
level	TEXT	Severity: WARN, ERROR, INFO
logger	TEXT	Component: payment.client, dns.resolver, kernel.net, infra.automation
message	TEXT	Log message text
attributes	JSONB	Additional context: pod, node_pool, service, change_type, etc.

Column	Type	Description
tenant_id	TEXT NOT NULL	Tenant identifier for multi-tenant isolation
ts	TIMESTAMPTZ NOT NULL	Metric collection timestamp
metric_name	TEXT NOT NULL	e.g., checkout_success_rate, dns_query_latency_p99_ms, kernel_rx_queue_latency_ms
value	DOUBLE PRECISION	Numeric metric value
labels	JSONB	Dimensions: region, cluster, node_pool, pod, service, query_type, etc.

Column	Type	Description

tenant_id	TEXT NOT NULL	Tenant identifier for multi-tenant isolation
ts	TIMESTAMPTZ NOT NULL	Metric collection timestamp
metric_name	TEXT NOT NULL	e.g., checkout_success_rate, dns_query_latency_p99_ms, kernel_rx_queue_latency_ms
value	DOUBLE PRECISION	Numeric metric value
labels	JSONB	Dimensions: region, cluster, node_pool, pod, service, query_type, etc.

4.4. Foundation Models

Role	Model ID	Region	Purpose
Supervisor (Primary LLM - Routing + Reasoning)	Global.anthropic.claude-sonnet-4-5-20250929-v1:0	ap-south-1	Primary model for agent reasoning, query classification, multi-agent orchestration, and RCA synthesis (as per SOW Section 3.16)
CloudWatch Specialist	Global.anthropic.claude-haiku-4-5-20251001-v1:0	ap-south-1	CloudWatch alarm/metric/log tool execution
Security Specialist	Global.anthropic.claude-haiku-4-5-20251001-v1:0	ap-south-1	Security Hub findings and compliance tool execution
Cost Specialist	Global.anthropic.claude-haiku-4-5-20251001-v1:0	ap-south-1	Cost Explorer analysis tool execution
Advisor Specialist	Global.anthropic.claude-haiku-4-5-20251001-v1:0	ap-south-1	Trusted Advisor recommendations tool execution
Jira Specialist	Global.anthropic.claude-haiku-4-5-20251001-v1:0	ap-south-1	Jira ticket management tool execution
Knowledge Specialist	global.anthropic.claude-haiku-4-5-20251001-v1:0	ap-south-1	AWS documentation retrieval tool execution
Investigator (RCA Agent)	Global.anthropic.claude-haiku-4-5-20251001-v1:0	Ap-south-1	SQL-based telemetry investigation, multi-step RCA on otel_metrics/logs/spansa

5. Agent Pipeline Design

5.1. Supervisor Agent

5.1.1. The Supervisor Agent serves as the entry point for all user queries. It classifies user intent, delegates requests to the appropriate specialist agent(s) using the A2A protocol, maintains conversational context through AgentCore Memory, and synthesizes responses when queries span multiple domains.

5.2. CloudWatch Agent

5.2.1. The CloudWatch Agent specializes in AWS CloudWatch observability. It retrieves metrics, alarms, log groups, and dashboards through the CloudWatch MCP Server and provides metric statistics, alarm states, log analysis, and observability insights.

5.3. Security Agent

5.3.1. The Security Agent focuses on AWS security posture by querying Security Hub, GuardDuty, and IAM configurations through the AWS API MCP Server. It identifies security findings, compliance gaps, overly permissive policies, and recommends remediation actions.

5.4. Cost Agent

5.4.1. The Cost Agent provides AWS cost analysis and optimization insights by accessing Cost Explorer, billing metrics, and resource utilization through the AWS API MCP Server. It detects cost anomalies, spending trends, and underutilized resources while recommending optimization opportunities.

5.5. Advisor Agent

5.5.1. The Advisor Agent retrieves AWS Trusted Advisor recommendations across cost optimization, security, fault tolerance, performance, and service limits. It prioritizes findings, identifies affected resources, and recommends corrective actions.

5.6. Jira Agent

5.6.1. The Jira Agent integrates incident management with Jira by creating, updating, and querying tickets during RCA workflows. It formats investigation findings into structured incident reports containing severity, affected services, root cause, and remediation recommendations.

5.7. Knowledge Agent

5.7.1. The Knowledge Agent retrieves AWS documentation, troubleshooting guides, and architectural best practices through the AWS Knowledge MCP Server. It enriches investigations with authoritative AWS guidance and contextual recommendations.

5.8. Investigator Agent

5.8.1. The Investigator Agent is the core Root Cause Analysis (RCA) engine. It directly queries the RDS PostgreSQL database using LangChain SQLDatabaseToolkit, correlates metrics, logs, and traces, executes an 8-step RCA workflow to identify anomalies and root causes, and

generates evidence-backed remediation recommendations while ensuring tenant isolation through tenant-specific filtering.

5.9. Session and Memory Design

5.9.1. **Short-Term Memory :** Short-term memory maintains the active conversation within a session by storing the complete message history. Each session is identified by a unique runtime session ID, while memory retrieval is scoped to the authenticated user using the Cognito actor ID to prevent cross-user access.

5.9.2. **Cross-Session Context :** Long-term memory stores Semantic Facts, which capture persistent user and environment information, and Session Summaries, which retain concise summaries of completed conversations. These enable the supervisor to recall previous interactions and continue investigations without requiring users to repeat context.

5.9.3. **Memory Namespace Isolation :** Memory is organized using user-specific namespaces following the /strategy/{strategy_id}/actor/{actorId}/ pattern, ensuring that semantic facts and session summaries remain isolated per authenticated user and preventing cross-user or cross-tenant context leakage.

6. Security

Security is implemented across all layers following AWS Well-Architected Framework security pillar best practices. The design enforces least-privilege access, encryption everywhere, and multi-tenant isolation at every boundary.

6.1. Identity and Access Management

Role	Purpose	Key Permissions
Backend-task-role	ECS Fargate task execution	Bedrock InvokeModel, AgentCore InvokeRuntime, DynamoDB RW, SecretsManager Read, CloudWatch Read, X-Ray Write
Backend-execution-role	ECS container pull and logging	ECR Pull, CloudWatch Logs Write
AgentCore Supervisor Role	Supervisor runtime execution	Bedrock InvokeModel, AgentCore InvokeRuntime (specialists), AgentCore Memory RW
AgentCore CloudWatch Role	CloudWatch specialist execution	ReadOnlyAccess, SecretsManager (msp-credentials/*), Bedrock InvokeModel

AgentCore Security Role	Security specialist execution	ReadOnlyAccess, SecretsManager (msp-credentials/*), Bedrock InvokeModel
AgentCore Investigator Role	RCA investigator execution	ReadOnlyAccess, SecretsManager (rca-db/*), RDS Describe, Bedrock InvokeModel

6.2. Authentication

6.2.1. Amazon Cognito User Pool ap-south-1_XoMze1ISf provides user authentication with client ID 7bs3ae73cnblm9o44io7vg4tk3. The authentication flow uses Cognito Hosted UI with authorization code grant for the React frontend. JWT tokens are validated on every API Gateway request using a Cognito Authorizer. The access token contains the user's sub claim which becomes the actorId for memory namespace isolation and tenant scoping throughout the agent pipeline.

6.2.2. Token refresh is handled client-side using Cognito refresh tokens with a 30-day expiry. Access tokens expire after 1 hour, requiring periodic refresh. The frontend stores tokens in secure HTTP-only cookies to prevent XSS-based token theft.

6.3. Bedrock Guardrails

6.3.1. Bedrock Guardrails are configured for INPUT validation only, applied to all agent invocations before the model processes the request. The guardrail filters block prompt injection attempts, harmful content categories (hate speech, violence, sexual content, self-harm), and personally identifiable information (PII) in user inputs. Content filtering thresholds are set to HIGH sensitivity for all categories.

6.3.2. The guardrail does not apply to OUTPUT because agent responses are deterministic and tool-driven they return factual observability data rather than generative content that could contain harmful material. This design choice reduces latency by eliminating output scanning while maintaining input safety.

6.4. Multi-Tenant Data Isolation

Multi-tenant isolation is enforced at five layers:

6.4.1. **Authentication** — Cognito JWT verified on every request; unauthenticated requests are rejected at API Gateway.

6.4.2. **Session** — Each session is scoped to actorId = user_cognito_sub preventing cross-user session access.

6.4.3. **Memory** — AgentCore Memory namespaces are partitioned as /strategy/{id}/actor/{actorId}/ ensuring no cross-tenant fact retrieval.

6.4.4. **AWS Access** — STS AssumeRole generates temporary credentials scoped to the customer's AWS account with 1-hour expiry.

6.4.5. **Runtime** — AgentCore microVM isolation ensures each invocation executes in a dedicated compute environment with no shared state.

6.5. Data Protection

6.5.1. Encryption at Rest:

Resource	Encryption Method	Details
S3 Buckets (frontend)	SSE-S3	Server-side encryption with Amazon S3-managed keys
S3 Buckets (AgentCore assets)	SSE-S3	Server-side encryption with Amazon S3-managed keys
DynamoDB (solution-chat-requests)	AWS-managed encryption	Encryption at rest enabled by default
RDS PostgreSQL (rca-scenario-db)	Not encrypted (poc)	StorageEncrypted: false — recommended for production
AgentCore Memory	AWS-managed encryption	Encrypted at rest by managed service
Secrets Manager	AWS KMS	All secrets encrypted with KMS
ECS Task Communication	TLS 1.2+	All AWS API calls over HTTPS
CloudFront → ALB	HTTPS only	Viewer protocol policy: redirect-to-https
ALB → ECS	HTTP (port 8000)	Internal VPC traffic (private subnets)
ECS → AgentCore	TLS 1.2+ (SigV4)	invoke_agent_runtime over HTTPS with SigV4 auth
ECS → RDS	TLS (sslmode=prefer)	PostgreSQL connection with SSL preferred
Agent → Tenant AWS	TLS 1.2+	STS AssumeRole + all API calls over HTTPS

6.5.2. Access Controls

All data in transit is encrypted using TLS 1.2 or higher. CloudFront enforces HTTPS-only with TLSv1.2_2021 security policy. API Gateway endpoints use AWS-managed certificates. Internal service communication between ECS and AgentCore uses SigV4-signed HTTPS requests. Database connections to RDS use SSL with certificate verification enabled.

6.5.3. Access Controls:

S3 bucket policies deny any request not using SSL (aws:SecureTransport: false). Security groups restrict RDS access to the ECS task security group and AgentCore runtime CIDR ranges.

DynamoDB access is controlled exclusively through IAM policies attached to the ECS task role. No IAM users or long-lived access keys exist in the account; all access uses IAM roles with temporary credentials.

7. Networking

The network architecture uses a combination of public and private endpoints appropriate for a solution deployment. All external-facing services are protected by WAF and CloudFront edge security.

7.1. Network Architecture

Component	Endpoint Type	Access Pattern
CloudFront	Public (Edge)	End-user browser access via HTTPS
API Gateway	Regional (Public)	CloudFront origin; Cognito-authorized
ECS Fargate	Private (VPC)	API Gateway integration via VPC Link
RDS PostgreSQL	Public (VPC)	AgentCore runtime access; SG-restricted
DynamoDB	Regional (AWS)	VPC endpoint preferred; IAM-authorized
AgentCore Runtime	Regional (AWS)	ECS backend via SDK; IAM-authorized
AgentCore Gateway	Regional (AWS)	Agent runtimes via SigV4 + OAuth
S3	Regional (AWS)	CloudFront OAI; IAM-authorized

7.2. Data Flow Security

User requests originate from the browser, hitting CloudFront edge locations where WAF rules (poc-waf) inspect for common web exploits (SQL injection, XSS, rate limiting). Valid requests pass to API Gateway where the Cognito authorizer validates the JWT token. Authenticated requests are forwarded to the ECS Fargate task running within the VPC. The FastAPI backend invokes AgentCore Runtime using IAM-signed SDK calls over HTTPS. AgentCore Runtime executes the supervisor agent which makes A2A calls to specialist runtimes, all within the AgentCore managed infrastructure. Specialist agents access external data sources (RDS, CloudWatch, Cost Explorer) using temporary credentials obtained via STS AssumeRole. Responses flow back through the same path, with DynamoDB storing the request/response pair for chat history before the final response reaches the user.

8. Logging and Monitoring

8.1. Comprehensive observability is implemented using CloudWatch, X-Ray, and AgentCore Observability to provide full visibility into system health, agent performance, and operational anomalies.

8.2. CloudWatch Metrics

CloudWatch collects metrics from all infrastructure components including ECS task utilization, API Gateway latency and error rates, DynamoDB consumed capacity, and RDS connection counts. Custom metrics are published for agent-specific KPIs including invocations per minute, average response time, token consumption, and cost per query.

8.2.1. Key metrics monitored:

- **8.2.1.1.** AgentInvocationCount — Total agent runtime invocations per minute, segmented by runtime name.
- **8.2.1.2.** AgentResponseLatency — P50, P90, P99 response time in milliseconds per agent runtime.
- **8.2.1.3.** TokensConsumed — Input and output token counts per invocation for cost tracking.
- **8.2.1.4.** AgentErrorRate — Percentage of invocations resulting in errors, segmented by error type.
- **8.2.1.5.** ECSCPUUtilization — Fargate task CPU utilization percentage.
- **8.2.1.6.** ECSMemoryUtilization — Fargate task memory utilization percentage.
- **8.2.1.7.** RDSConnectionCount — Active database connections to rca-scenario-db.
- **8.2.1.8.** APIGateway5XXRate — Server error rate on the poc-api gateway.

8.3. AWS X-Ray Distributed Tracing

8.3.1. Distributed tracing is implemented using the AWS Distro for OpenTelemetry (ADOT) sidecar container deployed alongside the FastAPI backend in ECS. The ADOT collector captures trace segments from the FastAPI application instrumented with the OpenTelemetry Python SDK. Traces propagate through API Gateway (which adds its own segment), into ECS, and through to AgentCore invocations. This provides end-to-end visibility of request latency breakdown, enabling identification of bottlenecks in the agent pipeline.

AgentCore Observability supplements X-Ray by capturing internal agent execution traces including individual tool calls, LLM inference durations, and memory read/write operations within each agent's execution graph.

8.4. Alerting Configuration

Alarm	Metric	Threshold	Action
HighAgentLatency	AgentResponseLatency P99	> 60,000 ms (60s)	SNS → Ops team notification
HighErrorRate	AgentErrorRate	> 10% over 5 min	SNS → Ops team notification
ECSHighCPU	ECSCPUUtilization	> 80% for 5 min	SNS → Ops team notification
ECSHighMemory	ECSMemoryUtilization	> 85% for 5 min	SNS → Ops team

			notification
RDSHighConnections	RDSConnectionCount	> 80% of max_connections	SNS → Ops team notification
APIGateway5XX	APIGateway5XXRate	> 5% over 5 min	SNS → Ops team notification
DDBThrottling	DynamoDB ThrottledRequests	> 0 over 1 min	SNS → Ops team notification
HighCostAnomaly	AWS Cost Anomaly Detection	> 20% daily increase	SNS → Finance notification

8.5. Logging Strategy

All application logs are centralized in CloudWatch Logs with structured JSON formatting. Log groups follow the naming convention /poc/motadata/{service-name} for consistent organization.

8.5.1. **ECS Backend** — FastAPI access logs and application logs at INFO level; ERROR and above trigger alerting.

8.5.2. **AgentCore Runtimes** — Agent execution logs including tool invocations, model calls, and decision points.

8.5.3. **API Gateway** — Access logs with request ID, latency, status code, and caller identity.

8.5.4. **CloudTrail** — Management events for all API calls across the account for audit and compliance.

Log retention is configured at 30 days for operational logs and 90 days for CloudTrail and security-related logs. Log Insights queries are pre-configured for common troubleshooting scenarios including agent timeout analysis, error pattern detection, and cost anomaly investigation.

9. Cost Optimization

The solution is designed for cost efficiency while maintaining production-grade architecture patterns. Cost optimization focuses on right-sizing compute, leveraging serverless services, and implementing model selection strategies.

9.1. Cost Optimization Strategies

9.1.1. **Model Tiering** — The supervisor uses Claude Sonnet 4.5 only for intent classification and response aggregation; all domain reasoning uses the more cost-effective Claude Haiku 4.5 (approximately 10x cheaper per token).

9.1.2. **Prompt Caching** — System prompts are cached across invocations, reducing input token costs by up to 90% on repeated prefixes.

9.1.3. **Serverless Execution** — AgentCore Runtime eliminates idle compute costs; charges are per-invocation only.

9.1.4. **Right-Sized Compute** — ECS Fargate task uses minimal resources (512 CPU / 1024 MB) appropriate for an API proxy workload.

9.1.5. **On-Demand DynamoDB** — Pay-per-request pricing avoids over-provisioning for variable solution workloads.

9.1.6. **db.t3.micro RDS** — Smallest available instance class suitable for poc telemetry data volumes.

9.1.7. **Single-AZ Deployment**— poc does not require multi-AZ redundancy, reducing RDS and networking costs.

9.1.8. **CloudFront Caching** — Static frontend assets served from edge cache reduce S3 request costs.

10. Development and Deployment Practices

Development follows a trunk-based workflow with automated deployment pipelines. All infrastructure is defined as code using AWS CDK and AgentCore CLI.

10.1. Code Management

The project repository `msp-ops-langgraph` is structured with clear separation between agent code, infrastructure definitions, backend API, and frontend application. Branch protection requires pull request reviews before merging to the main branch. Code changes follow a feature branch workflow with descriptive branch naming (`feature/`, `fix/`, `docs/`).

10.1.1. **Key directories:**

10.1.1.1. **agents/runtime/** — Supervisor agent (LangGraph + FastAPI)

10.1.1.2. **agents/runtime_*_langgraph/** — Specialist agents (7 agents)

10.1.1.3. **agentcore-deploy/motadatamsp/** — CDK project for AgentCore deployment

10.1.1.4. **backend/** — FastAPI application with ECS Dockerfile

10.1.1.5. **frontend/** — React 18 application

10.1.1.6. **scripts/** — Operational and demo scripts

10.2. Deployment Process

10.2.1. **Backend deployment follows a containerized workflow:**

10.2.1.1. Build the Docker image with `--platform linux/amd64` for Fargate compatibility.

10.2.1.2. Push to ECR repository `001961766007.dkr.ecr.ap-south-1.amazonaws.com/poc-backend`.

10.2.1.3. Force new deployment on ECS service `poc-backend-service` in **poc-cluster**.

10.2.1.4. ECS performs rolling update with health check validation before draining old tasks.

10.2.2. **Frontend deployment:**

10.2.2.1. Build the React application with `npm run build`.

10.2.2.2. Sync built assets to the S3 frontend bucket.

10.2.2.3. Create CloudFront invalidation to purge cached assets.

10.3. AgentCore Deployment

10.3.1. Agent runtimes are deployed using the AgentCore CLI with CDK-defined configurations

10.3.1.1. Export AWS credentials: `eval $(aws configure export-credentials --profile motadata --format env)`

10.3.1.2. Navigate to CDK project: `cd agentcore-deploy/motadatamsp`

10.3.1.3. Deploy all runtimes: `agentcore deploy --yes`

The CDK project defines all 11 runtimes (1 supervisor + 7 specialist A2A + 3 MCP servers) with their respective configurations including model IDs, environment variables, memory associations, and gateway connections. Runtime updates are zero-downtime — AgentCore provisions new runtime instances before draining old ones.

11. Data Operations

Data operations cover the ingestion of telemetry data into the RDS database and the refresh process for demo scenarios used in testing and demonstrations.

11.1. Telemetry Data Ingestion

Telemetry data is ingested into the RDS PostgreSQL database using the `dump_scenario_postgres.sh` shell script which generates and loads OpenTelemetry-format scenario data directly via `psql`. The script:

11.1.1.1. Generates the complete RCA scenario data (metrics, logs, traces) using embedded Python with timestamps relative to execution time.

11.1.1.2. Connects to the PostgreSQL database via `psql` client and executes bulk INSERT statements within a transaction.

11.1.1.3. Supports multi-tenant ingestion via the `--tenants` flag, inserting identical scenario data per tenant with the appropriate `tenant_id`.

11.1.1.4. Provides idempotent re-runs via the `--truncate` flag which deletes existing tenant data before reinsertion.

Per-tenant data volumes: 12,120 metric rows, 286 log rows, and 840 span rows for the default 60-minute time window.

12. Data Governance

Data governance ensures proper handling, classification, and lifecycle management of all data within the solution environment.

12.1. Multi-Tenancy

All data storage layers enforce tenant isolation. The RDS database uses `tenant_id` as a mandatory column on every telemetry table, and all agent-generated SQL queries include a `WHERE tenant_id = :tenant` clause enforced by the system prompt. DynamoDB partitions chat history by `user_id` derived from the Cognito JWT. AgentCore Memory namespaces partition semantic facts and session summaries per authenticated user. No cross-tenant data access is possible without explicit IAM role assumption for the target account.

12.2. Data Classification

Data Type	Classification	Protection
Tenant AWS credentials (STS tokens)	Sensitive	Secrets Manager, 1-hour expiry, auto-refresh
RDS database credentials	Sensitive	Secrets Manager (rca-db/*), never logged
Tenant telemetry data (metrics/logs/traces)	Confidential (per-tenant)	tenant_id row isolation in PostgreSQL
Cognito JWT tokens	Internal	Short-lived (1 hour), HTTPS only
AgentCore Memory (conversation)	Internal	Encrypted at rest, scoped by actorId
Chat request state (DynamoDB)	Internal	Encrypted at rest (AWS-managed)
Frontend assets (S3)	Public	CloudFront distribution, no sensitive data

12.3. Data Lifecycle

12.3.1. Data lifecycle policies ensure that data does not persist beyond its useful operational window:

12.3.1.1. Chat History— 30-day TTL configured on DynamoDB with automatic deletion via TTL expiry.

12.3.1.2. Telemetry Data — 90-day retention in RDS; older data is archived to S3 Glacier or deleted per tenant preference.

12.3.1.3. Application Logs — 30-day retention in CloudWatch Logs; critical security logs retained for 90 days.

12.3.1.4. CloudTrail Logs — 90-day retention for compliance and audit requirements.

12.3.1.5. Frontend Assets — Retained indefinitely in S3 with versioning; old versions expire after 30 days.

12.3.1.6. RCA Results — Archived to S3 for 1 year; accessible for historical investigation review.

12.4. Access Controls

12.4.1. Data access is governed exclusively through IAM roles — no IAM users or long-lived credentials exist in the account. Access control follows these principles:

12.4.1.1. Service-to-service communication uses IAM roles with minimum required permissions.

12.4.1.2. Human access to production data requires AWS SSO with MFA and is restricted to the operations team.

12.4.1.3. Database access is restricted via security groups; no direct internet access to RDS is permitted for production workloads.

12.4.1.4. S3 bucket policies enforce encryption in transit and deny any non-SSL requests.

12.4.1.5. All data access is logged via CloudTrail for audit and compliance review.

13. Disaster Recovery and Resilience

The solution environment implements basic resilience features appropriate for a proof-of-concept deployment. The design prioritizes availability within a single region while documenting the path to production-grade disaster recovery.

13.1. Recovery Objectives

13.2. Recovery Process

In the event of a service disruption, the recovery process follows this sequence:

13.2.1. Detection — CloudWatch Alarms trigger within 5 minutes of threshold breach; SNS notifies the operations team.

13.2.2. Assessment — Operations team evaluates alarm context, X-Ray traces, and CloudWatch Logs to determine impact scope.

13.2.3. Recovery — For ECS failures, force new deployment triggers automatic task replacement. For RDS failures, point-in-time recovery restores from the latest automated backup. For AgentCore failures, runtime restart is triggered via CLI.

13.2.4. Validation— End-to-end test query is executed to confirm system functionality post-recovery.

13.2.5. Post-Incident — Root cause analysis of the outage is documented and preventive measures are implemented.

13.3. Resilience Features

The following resilience features are implemented in the current solution deployment:

13.3.1. ECS Service Recovery — ECS automatically replaces unhealthy tasks based on health check failures; minimum task count of 1 ensures service availability.

13.3.2. DynamoDB High Availability — DynamoDB automatically replicates across multiple AZs within the region with no configuration required.

13.3.3. RDS Automated Backups — Daily automated snapshots with 7-day retention and point-in-time recovery capability.

13.3.4. AgentCore Managed Resilience — AgentCore Runtime handles runtime failures internally with automatic restart and retry logic.

13.3.5. CloudFront Edge Caching — Static frontend assets remain available from edge cache even during origin failures.

13.3.6. Graceful Degradation — The ECS backend implements circuit breaker patterns; if AgentCore is unavailable, the system returns a service-unavailable response rather than hanging indefinitely.

13.3.7. Infrastructure as Code — All resources are defined in CDK and can be redeployed from scratch in a new account/region within 2 hours.